

Working With Memory in LangChain

Introduction to Memory

Large Language Models are stateless by design, meaning each message sent to a model is processed independently. The model does not have access to previous messages in the conversation history.

To see how this stateless behavior in language models affects a conversation, consider the following sequence of messages sent to a model:

Message 1:

User: My name is Maya, and I'm planning a trip to Japan.

Model: That sounds exciting, Maya! Japan is wonderful.

Message 2:

User: What are some must-see places there?

Model: I'd be happy to provide you with recommendations! Where exactly are you referring to?

The model cannot use information from Message 1 when responding to Message 2. As a result, it lacks the context that the user is referring to Japan, and each message is treated independently.

To enable context-aware conversations, LangChain introduces a memory component.

Memory in LangChain stores the sequence of messages that make up the conversation history. This history is attached to each request sent to the model, enabling the model to process the current message alongside previous messages in the conversation.

Let's see how the same conversation behaves differently with memory enabled:

Message 1:

User: My name is Maya, and I'm planning a trip to Japan.

Model: That sounds exciting, Maya! Japan is wonderful.

Message 2:

User: What are some must-see places there?

Model: Since you're planning a trip to Japan, some must-see places include Tokyo, Kyoto, and Mount Fuji.

Here, with memory enabled, the model can reference information from Message 1 when answering Message 2 because it now has access to the stored conversation history.

To enable context-aware conversations, memory relies on a simple cycle that runs for each request:

1. **Retrieve**: Before each user request is processed by the model, the stored conversation history is loaded.
2. **Combine**: The loaded conversation history is then merged with the user's new request so the model can consider both previous and current messages when generating a response.
3. **Process**: The model receives both the conversation history and the current user request together, allowing it to generate a context-aware response.
4. **Store**: After the model response is generated, the new exchange (current user request + model response) is saved in memory for future requests.

This cycle enables memory to preserve context across conversations. However, storing all messages is not always practical, especially as conversations grow longer. To address this, LangChain offers various memory types to manage conversation history, tailored to different application needs.

Memory Type	How It Works
Buffer	Stores all messages exactly as written
Window	Keeps only the last N exchanges
Summary	Condenses older messages into summaries
Entity	Extracts and stores key facts separately

In this lesson, we'll implement these memory types and learn how to:

- Implement memory using LangChain components.
- Measure and diagnose token usage.
- Choose the right memory strategy for different use cases.
- Combine strategies for advanced applications.

Implementing Your First Memory

Now that we understand how memory operates, we'll build a personal assistant that preserves conversation history. We'll use buffer memory to get started.

Buffer memory is the simplest memory strategy — it stores every user message and model response exactly as they occur, without summarizing, filtering, or removing. This makes it an ideal starting point for understanding how memory integrates into a LangChain application.

Implementing buffer memory requires handling two responsibilities:

- storing conversation messages
- attaching those messages to each model request automatically

LangChain handles these responsibilities using two components.

- **InMemoryChatMessageHistory**: This component stores conversation messages in memory. It keeps a sequenced list of user messages and model responses for an active conversation.
- **RunnableWithMessageHistory**: This component tracks chat history during each model request. It automatically stores and retrieves messages before passing them to the model.

Together, these components enable session-based memory management, where each conversation is identified by a unique session ID that serves as a label to keep messages from different conversations separate.

How Session-Based Memory Works

An application can run multiple conversations simultaneously. Each conversation is assigned a unique session ID that remains constant throughout the conversation. Every message in the conversation includes its session ID, enabling the application to store messages correctly and retrieve the corresponding conversation history.

Example: how session IDs keep conversations separate

Imagine an application where one user is running two separate conversations at the same time.

Conversations start

- Conversation A is assigned a session ID: session_101.
- Conversation B is assigned a session ID: session_202.

Each session ID represents a separate conversation.

Messages arrive

Conversation A

- (session_101) User message: "I'm planning a trip to Italy."
- (session_101) Assistant response: "That sounds exciting!"

These messages are stored under session_101.

Conversation B (session_202)

- (session_202) User message: "I am creating a project on volcanoes."
- (session_202) Assistant response: "That's an interesting topic."

These messages are stored under session_202.

A new message is sent.

Later, the user in Conversation A sends another message:

(session_101) User message: "Can you recommend some must-see places?"

Because this message arrives with session_101, the application:

- loads the history for Conversation A
- sees the earlier message about Italy
- uses that context to generate a response

Assistant: "Since you're planning a trip to Italy, some must-see places include Rome and Venice."

The new user message and the assistant's response are saved back under session_101.

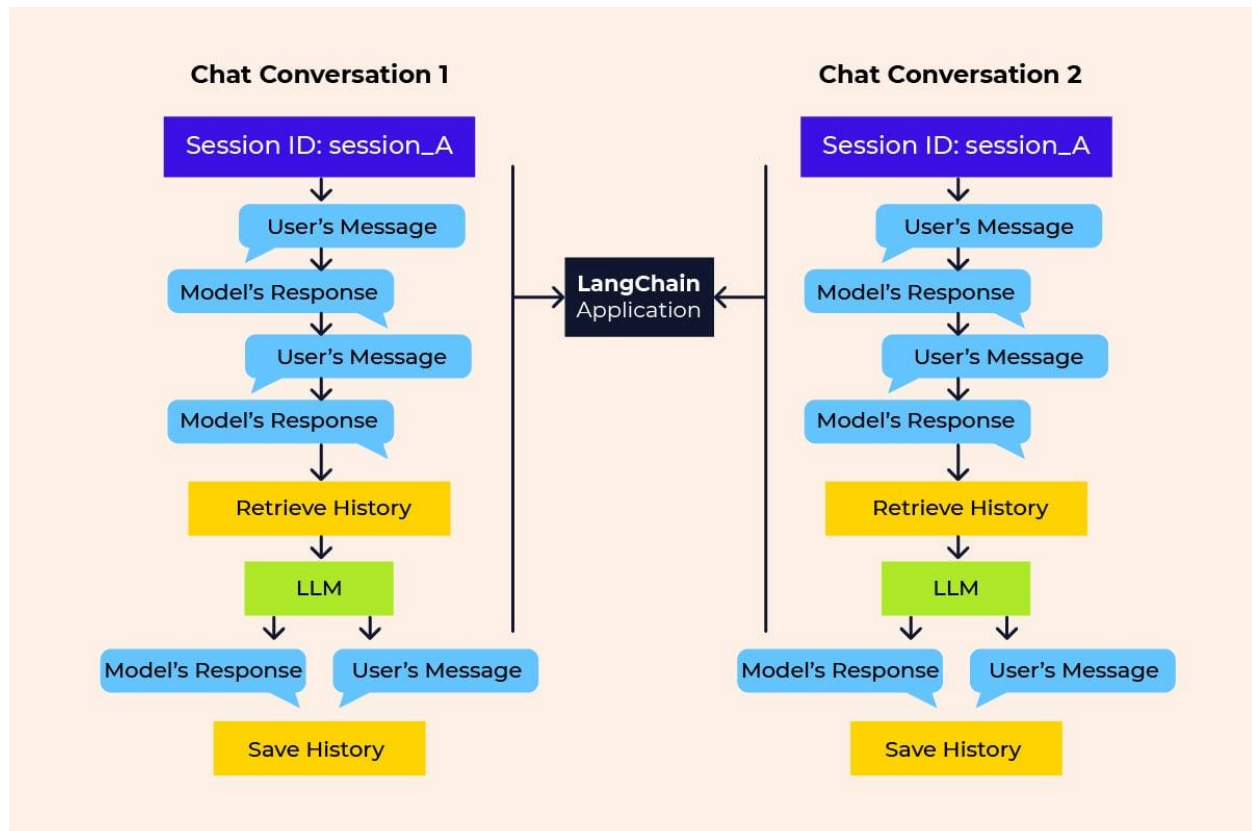
Conversations stay separate

Conversation B continues independently under session_202. Messages from Conversation A are never mixed with Conversation B because each session ID maintains its own conversation history.

Key Takeaway

Session IDs act as labels for conversations. They ensure that:

- Each conversation has its own continuous history.
- Multiple conversations can run at the same time without interfering with one another.



Note: Buffer memory does not scale well for long conversations. Since every message is retained, token usage increases with each turn. For extended conversations, other memory strategies, such as window or summary memory, are more efficient.

To add buffer memory to the application, we'll:

- Create a session history function.
- Build a prompt template that includes conversation history.
- Wrap the chain with memory management.
- Test the conversation flow.

Instructions

1. Checkpoint 1 Passed

1.

Creating the Session History Function

A dictionary named `store` and a function named `get_session_history()` are defined to manage chat history across different sessions. The function takes a `session_id` parameter and returns the corresponding chat history from the store.

Check if the `session_id` exists in the store dictionary. If it doesn't exist, create a `InMemoryChatMessageHistory` object and add it to the store with the `session_id` as the key.

2. Checkpoint 2 Passed

2.

Creating a Conversational Prompt Template

Use the `ChatPromptTemplate` class to create a prompt template named `prompt` that includes conversation history. This template includes three components:

- Add a system message that defines the assistant's role and behavior.
- Add a `MessagesPlaceholder` with `variable_name` set to "history" for storing past conversation messages.
- Add a human message template with an input placeholder for the current user message.

Create a `base_chain` that connects the prompt template, the language model, and a string output parser so each component passes its output to the next.

3. Checkpoint 3 Passed

3.

Wrapping the Chain With Message History

At this point, we have message storage in place and a prompt that accepts conversation history. Now, use `RunnableWithMessageHistory` to wrap the conversation chain and enable automatic history handling. This ensures that stored messages are retrieved before each request and updated after each response.

Save the wrapped chain in a variable named `conversation_chain`.

Pass the following arguments:

- `base_chain` as the runnable to wrap
- `get_session_history` to retrieve or create chat history for each session
- `input_messages_key` set to "input" to indicate where the user's current message is provided
- `history_messages_key` set to "history" to indicate where past conversation messages are stored

Next, define a session configuration by creating a nested dictionary with a "configurable" key that contains a "session_id" set to "user-101". Assign this configuration to a variable named `session_config`. This determines which conversation history is used when invoking the chain.

4. Checkpoint 4 Passed

4.

Displaying the Conversation History

Retrieve the chat history for the session by calling the `get_session_history()` function with "user-101" as the session ID and store the result in a variable named `session_history`.

The provided loop iterates through all messages in the session history, assigns a role label "User" or "Assistant" based on the message type, and prints each message with its number, role, and content.

Instructions

Examine the memory cycle diagram. The diagram shows how conversation history flows through the memory cycle. When you're ready, select "Next" to continue implementing Buffer Memory, the simplest approach that stores everything.

Diagnosing Token Limit Problems

Buffer memory stores every message from a conversation and sends the entire message history to the model with every new request, as part of the input.

As a conversation grows, each request sent to the model becomes larger because more message history is included in the input context. This growth is measured in **tokens** — the

units of text the model uses to process a request. A short word may correspond to one token, while a longer or compound word may be split into multiple tokens.

Token Growth With Buffer Memory

A conversation is made up of multiple turns. Each conversation turn is a single completed exchange that adds one user message and one model response. A single turn typically contributes 50 – 150 tokens to the conversation.

Every request to the model includes:

- System instructions (~50 – 200 tokens)
- Conversation history from memory (grows with each turn)
- Current user input (~20 – 100 tokens)
- Space reserved for the model’s response (~100 – 500 tokens)

Note: The token values here are estimated and not fixed. Actual token usage depends on the content of the messages.

The example table below illustrates how token usage increases as more turns are added to the conversation history included in a single request.

Conversation Length	Approximate Token Usage	What the Single Request Contains
1 turn	~50 – 150 tokens	System instructions + 1 completed turn (1
5 turns	~250 – 750 tokens	System instructions + 5 completed turns (5
20 turns	~1,000 – 3,000 tokens	System instructions + 20 completed turns
50 turns	~2,500 – 7,500 tokens	System instructions + 50 completed turns
100 turns	~5,000 – 15,000 tokens	System instructions + 100 completed turns

Note: Each request includes the accumulated conversation history and the user message being processed. This increasing request size is not unlimited.

As requests grow in token count, token limits become an important constraint to diagnose.

The token limit constraint is enforced by the model’s context window.

A **context window** defines a limit on the maximum number of tokens the model can process in a single request. If the total number of tokens in a single request exceeds the defined limit, the model cannot process the request and fails to generate a response.

Context Window Size

Context window sizes vary by model and may change over time.

The following values are examples:

- GPT-3.5-turbo: approximately 16,000 tokens
- GPT-4: approximately 128,000 tokens
- GPT-4o-mini: approximately 128,000 tokens

Note: These limits are not fixed. Context window sizes can change as models are updated or replaced. Always check the [OpenAI Models documentation](#) for the model used.

The problem is not the buffer memory itself. The problem is that requests keep getting larger. Larger requests exceed the context window and cause token overflow.

Fixing the Problem With Window Memory

One way to prevent token overflow is to use **window memory**.

Instead of keeping the entire conversation, window memory stores only the most recent k exchanges. The value of k is chosen based on how much recent context is needed in the conversation.

When a new message arrives, and the window is full, older exchanges are removed. This maintains a constant request size, even as the conversation progresses.

Trade-off: Older context is lost.

Benefit: Requests stay within token limits, and conversations can continue indefinitely.

In this exercise, we will start by loading a preconfigured LangChain conversation with buffer memory and session-based message storage. This setup will allow us to observe how request size grows as the conversation history accumulates, without having to build the chain manually.

We will then:

- Measure token growth with buffer memory.
- Identify when requests exceed the context window.
- Apply window memory.
- Verify that request size remains stable.

This workflow will mirror how token limit issues are diagnosed and fixed in real production systems.

Instructions

1. Checkpoint 1 Passed

1.

Running a Multi-Turn Support Conversation With Buffer Memory

Run through a complete support conversation to see buffer memory in action as it retains every message exchanged.

1. Define a session configuration dictionary named `buffer_config` that specifies the "session_id" as "support_buffer" to create a unique session identifier that isolates the memory storage for this conversation.
2. A loop structure with `print()` statements is provided to run through the conversation and display each turn.
3. Inside the loop, invoke the `conversation_chain` with the current turn as "input" and the session configuration, storing the result in the response variable so each message is sent to the chain while maintaining session context for conversation continuity.

2. Checkpoint 2 Passed

2.

Analyzing Buffer Memory Usage

Take a closer look at what's stored in buffer memory and calculate how much it grows with each conversation turn.

1. Retrieve the chat history for the "support_buffer" session and store the messages in a variable named `buffer_messages` to access the complete conversation history stored in memory for analysis.
2. Join all message contents into a single string using spaces and store it in a variable named `buffer_content` to create a single text representation that enables token estimation.
3. Estimate the token count by multiplying the number of words in `buffer_content` by 1.3 to get an approximate token count, since tokens don't map one-to-one with words. Store the result in a variable named `buffer_tokens`.

The print() statements are provided to display the buffer memory analysis results.

3. Checkpoint 3 Passed

3.

Implementing the Window Memory Trim Function

Create a utility function that keeps conversations manageable by automatically trimming older messages beyond a set limit.

A constant WINDOW_SIZE is set so the chat keeps only the last 5 exchanges (10 messages total).

A function called trim_to_window is also defined, which takes two parameters:

- history (the chat history object)
- k (the number of exchanges to keep, defaulting to WINDOW_SIZE)

Inside the function:

3. Check if the total number of messages in the history exceeds the window limit.
4. If it does, extract only the most recent messages within the limit and store them temporarily in a variable named recent_messages to preserve the conversations that should be retained.
5. Use the .clear() method to clear the entire history.
6. Create a nested loop to add back only the extracted recent_messages, maintaining the sliding window of conversation context.

4. Checkpoint 4 Passed

4.

Running a Multi-Turn Support Conversation With Window Memory

Run the same support conversation using window memory to see how it keeps only recent exchanges while discarding older ones.

1. Define a session configuration dictionary named window_config that specifies the "session_id" as "support_window" to create a unique session identifier that isolates the memory storage for this conversation.
2. A loop structure with print() statements is provided to run through the conversation and display each turn.

3. Inside the loop, invoke the `conversation_chain` with the current turn as "input" and the session configuration, storing the result in the response variable so each message is sent to the chain while maintaining session context.
4. After each invocation, call the `trim_to_window()` function with the session's history from the store and `WINDOW_SIZE` to automatically remove older messages and maintain the sliding window of recent conversations.
 - Make sure to nest the `session_id` inside both "configurable" and "session_id" keys in the `window_config` dictionary.
 - Make sure the config parameter is separate from the input dictionary, and that you pass it as a second argument to the `.invoke()` method.
 - Make sure to pass the session from the store dictionary using the key "support_window" as the first argument to `trim_to_window()`.
 - Use the `k` parameter in `trim_to_window()` to specify the window size, setting it equal to `WINDOW_SIZE`.
5. Checkpoint 5 Passed

5.

Analyzing Window Memory Usage

Check what's left in window memory after trimming to see how it maintains stable token usage throughout the conversation.

1. Retrieve the chat history for the "support_window" session and store the messages in a variable named `window_messages` to access the limited conversation history maintained by the sliding window.
2. Join all message contents into a single string using spaces and store it in a variable named `window_content` to create a single text representation that enables token estimation.
3. Estimate the token count by multiplying the number of words in `window_content` by 1.3 to get an approximate token count since tokens don't map one-to-one with words. Store the result in a variable named `window_tokens`.

The `print()` statements are provided to display the window memory analysis results.

- Make sure to access the `messages` attribute using a dot after calling the `get_session_history()` function to retrieve the list of message objects.

- Use a list comprehension with the content attribute using a dot to extract text from each message object.
- Use the .join() method with a space " " to combine all message contents into a single string.
- Use the len() function on the result of the .split() method to count the number of words in the content.
- Make sure to multiply the word count by 1.3 to get the rough token estimate.

Choosing the Right Memory Type

In the previous exercise, we used window memory to prevent token overflow by limiting the amount of conversation history included in each request sent to the model. This allowed conversations to continue without exceeding the model's token limit.

However, important information provided earlier in the conversation was lost when older messages were removed. This raises a question: *"How can we preserve important information while still managing token limits?"*

Memory is not just storage. It is a mechanism that determines which past messages need to be reused when the model responds. As conversations grow, different memory strategies reuse conversation history in different ways, leading to different trade-offs in context retention and token usage.

- Buffer memory reuses all past messages, causing token usage to grow with each turn and increasing the risk of exceeding the model's token limit.
- Window memory keeps only recent exchanges. Older messages are removed, reducing token usage but losing earlier context.
- Summary memory keeps a condensed representation of older context. Exact wording is removed, but the general meaning is preserved.

Among these strategies, summary memory balances context preservation with controlled token usage.

How Summary Memory Works

Instead of discarding older messages, summary memory condenses them into brief summaries as the conversation progresses. These summaries preserve general context and important information while removing repetition, examples, and exact wording.

Each request sent to the model then includes:

- a high-level summary of the earlier conversation
- the most recent detailed messages

This approach keeps token usage low while maintaining awareness of earlier exchanges.

Note: The trade-off is that exact wording and fine-grained details may be lost during summarization.

Choosing the Right Memory Type

There is no single best type of memory. The appropriate choice depends on the application's requirements.

Requirement	Recommended Memory Type
Conversations are short (under 10 turns).	Buffer
Conversations are long or run indefinitely.	Window or Summary
Every detail from the conversation must be preserved.	Buffer
General awareness of past context is sufficient.	Summary
Recent context matters more than older context.	Window
Older context matters, but exact wording does not.	Summary

Examples

- A quick Q&A bot with 3 – 5-turn conversations → Buffer (token growth is minimal)
- A coding assistant debugging a current error → Window (recent context matters most)
- A therapy companion used over multiple sessions → Summary (long-term awareness without high token cost)
- A legal assistant where exact phrasing matters → Buffer (no information can be lost or reworded)
- A customer support bot handling complex issues → Window or Summary (depends on whether older details are referenced)

In this exercise, we will run the same short conversation through all three memory types. The conversation consists of a fixed sequence of user questions about a dessert recipe, including dietary restrictions, preferences, and preparation details.

Using the same conversation each time allows us to observe how different memory strategies behave under identical conditions by:

- applying summary memory alongside buffer and window memory
- comparing how much conversation history each memory type retains
- checking whether each memory type can recall information mentioned earlier in the conversation

Instructions

1. Checkpoint 1 Passed

1.

Creating a Conversation Summary

Condense the full conversation into a brief summary by asking the LLM to extract key points.

A function named `create_summary()` is already defined with a `messages` parameter and returns the summary content. Inside the function:

2. Join all messages into a single string where each message includes its type and content on separate lines, storing this in a variable named `conversation_text` to create a readable representation of the entire conversation.
3. Create a summary prompt by combining an instruction to summarize the conversation with the conversation text, storing this in a variable named `summary_prompt`.
4. Invoke the summarizer model with the summary prompt wrapped in a `SystemMessage` and store the result in a variable named `summary` to generate the condensed version of the conversation.
 - Use the `.join()` method with a newline `"\n"` to combine all messages into a single string with each message on a separate line.
 - Use a list comprehension to format each message with an f-string that includes both the type and content attributes using dots.

- Use an f-string to create the summary prompt by combining the instruction text with the `conversation_text` variable.
- Make sure to wrap the `summary_prompt` inside a `SystemMessage` object before passing it to the `.invoke()` method.
- Use the `.invoke()` method on the summarizer model, passing a list containing the `SystemMessage` as an argument.
- Make sure to access the `content` attribute using a dot from the summary result to return only the text content.

2. Checkpoint 2 Passed

2.

Managing Summary Memory

Replace the full conversation history with a concise summary once enough exchanges have occurred.

A function named `manage_summary()` is already defined with a `history` parameter (the chat history object). Inside the function:

1. Check if the history contains at least two messages (one complete exchange) to ensure there's enough content worth summarizing.
2. If it does, create a summary of all messages in the history by calling the `create_summary()` function and store the result in a variable named `summary`.
3. Use the `.clear()` method to remove all messages from the history.
4. Add a new AI message to the message history, containing the conversation summary so the condensed context replaces the full message history.

3. Checkpoint 3 Passed

3.

Running the Conversation With Summary Memory

Run through a recipe conversation to see how summary memory condenses past exchanges into a brief context instead of storing everything.

1. Define a session configuration dictionary named `config_summary` that specifies the `"session_id"` as `"recipe_summary"` to create a unique session identifier for this conversation.

2. A loop structure with `print()` statements is already provided to run through the test messages and display selected turns.
3. Inside the loop, invoke the `conversation_chain` with the current message as input and the session configuration, storing the result in the response variable so each message is sent to the chain while maintaining context.
4. After each invocation, call the `manage_summary()` function with the session's history from the store to automatically replace the full conversation history with a condensed summary.

Conditional `print()` statements are provided to display Turn 2 in full (showing the allergy mention) while other turns are marked with a checkmark for brevity.

4. Checkpoint 4 Passed

4.

Comparing All Three Memory Types

See how buffer, window, and summary memory differ by comparing what each one stores after the same conversation.

1. A dictionary named `sessions` is already defined that maps session IDs to descriptive labels for each memory type.
2. A loop structure with `print()` statements is provided to display the comparison results.
3. Inside the loop, retrieve the messages for the current session by calling `get_session_history()` with the `session_id` and store the messages in a variable named `msgs` to access what each memory type has retained.

5. Checkpoint 5 Passed

5.

Testing Memory Recall

Ask the same question across all three memory types to see how each one uses its stored context to answer.

1. A question is already defined, and a dictionary named `config` maps descriptive labels to their corresponding session configurations.
2. A loop structure with `print()` statements is provided to display the question and responses from each memory type.

3. Inside the loop, invoke the `conversation_chain` with the recall question as input and the current configuration, storing the result in a variable named `response` to get each memory type's answer based on what it has retained.
 - Make sure the config parameter is separate from the input dictionary, and that you pass it as a second argument to the `.invoke()` method.
 - Make sure to use the `recall_question` variable as the value for the "input" key in the input dictionary.

Entity Extraction Memory

Memory strategies determine how conversation history is stored and included in each request. As conversations grow, token usage increases and can reach the model's context window limit.

To stay within the context window limit, some memory strategies trim, summarize, or discard older messages in the conversation. As a result, specific facts required from earlier messages are lost in the conversation. To prevent the loss of the required facts, we can use an entity extraction memory.

What Is Entity Extraction Memory?

Entity extraction memory uses a language model to identify and extract key information from messages in the conversation, then stores it as structured data in predefined fields. These fields remain available even if older messages are trimmed, summarized, or discarded.

To understand how entity extraction memory works, let's consider a healthcare conversation.

A user sends a message:

"My name is Sarah, and I'm allergic to dust."

We process this message through the following steps.

Step 1: Extract Facts From the Message

First, send a prompt to the language model with instructions about what information to extract.

```
prompt = f"""Extract patient information as JSON:
```

- patient_name
- medications
- allergies
- symptoms

```
Text: "My name is Sarah, and I'm allergic to dust."
```

```
Return only valid JSON."""
```

```
response = extractor.invoke([SystemMessage(content=prompt)])
```

Copy to Clipboard

The model processes the text and returns structured data identifying the facts:

- patient_name: ["Sarah"]
- allergies: ["dust"]

Step 2: Store Extracted Values

Next, the extracted information is stored in predefined entity fields. Each field maintains its value across conversation turns.

```
entity_memory = {  
    "patient_name": ["Sarah"],  
    "medications": [],  
    "allergies": ["dust"],  
    "symptoms": []  
}
```

At this point, the system has captured the key facts from the user's message in a structured format. Even if the original message is later trimmed or removed from the conversation history, these extracted values remain available.

Step 3: Add New Information in a Later Message

The conversation continues. The user sends another message:

“I’ve been having headaches for the past two days, and I’ve been taking aspirin.”

This message is processed using the same extraction step. The model identifies the new facts:

- symptoms: ["headache"]
- medications: ["aspirin"]

The new values get added to the existing entity memory.

The updated entity memory now looks like this:

```
{  
  "patient_name": ["Sarah"],  
  "medications": ["aspirin"],  
  "allergies": ["dust"],  
  "symptoms": ["headache"]  
}
```

Copy to Clipboard

Step 4: Use Entity Memory to Continue the Conversation

The user now asks:

“What should I avoid?”

When this question is received, the system retrieves the entity memory built so far. The current user message and the stored entity memory are included together when calling the model.

```
system_prompt = f"""You are a healthcare assistant.
```

Patient Information:

```
{json.dumps(entity_memory, indent=2)}
```

```
Use this information to answer the user's question."""
```

```
response = conversation_chain.invoke(  
    {"input": "What should I avoid?"}  
)
```

Because the model has access to the patient's allergies and symptoms from entity memory, it can generate a response based on those facts, even if the original messages are no longer in the conversation history.

For example, the model may respond:

“Since you are allergic to dust and experiencing headaches, you should avoid dusty environments.”

Step 5: Update an Existing Entity Field

Later in the conversation, the user adds more information:

“I’m also allergic to pollen, and I take metformin.”

The model extracts:

- allergies: ["pollen"]
- medications: ["metformin"]

These new values are added to the existing entity memory.

The entity memory now contains:

```
{  
  "patient_name": ["Sarah"],  
  "medications": ["aspirin", "metformin"],  
  "allergies": ["dust", "pollen"],  
  "symptoms": ["headache"]  
}
```

Copy to Clipboard

After this update, any future questions will use the latest stored information without reprocessing earlier messages.

For example, if the user later asks:

“What medications am I taking?”

The model can correctly respond:

“You are taking aspirin and metformin.”

When to Use Entity Memory

Entity memory is useful when:

- Specific facts must be retained across long conversations.
- Users provide personal details that should not be forgotten.
- The application needs to reference information mentioned many turns ago.
- Facts need to persist independently of conversation flow.

In this exercise, we add entity memory to a healthcare conversation to extract and store patient information.

Instructions

1. Checkpoint 1 Enabled

1.

Implementing Patient Entity Extraction

Use an LLM to identify and structure key patient information from the conversation text into a standardized JSON format.

A function named `extract_patient_entities()` is already defined with a text parameter to accept the conversation content for extraction. The test code is provided to verify that the extraction works correctly.

Inside the function:

1. Create a prompt that instructs the model to extract patient information as JSON with these exact fields: `patient_name`, `medications`, `allergies`, `symptoms`, `primary_doctor`, `insurance`, and `contact_info`.
2. Include the input text in the prompt and specify that empty lists or objects should be used if no information is found. Store this in a variable named `prompt` to provide clear extraction guidelines to the LLM.
3. Invoke the extractor model with the prompt wrapped in a `SystemMessage` to generate the structured extraction, and store the result in a variable named `response`.
4. Extract the content from the response and remove any Markdown wrapper by calling `strip_markdown_wrapper()`. Store the cleaned content in a variable named `content` to prepare it for JSON parsing.

A try-except block is provided to parse the content as JSON and return the result, with error handling for failed parsing.

2. Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Updating the Entity Store

Merge newly extracted patient information into the existing record without creating duplicate entries.

A function named `update_entity_store()` is defined with parameters for `session_id` and `extracted_entities` to merge new information into the existing patient record.

Inside the function:

1. Retrieve the existing patient record by calling `get_entity_store()` with the `session_id` and store it in a variable named `record` to access the current patient data.
2. Loop through the single-value fields (`patient_name`, `primary_doctor`, `insurance`). For each field, check whether it exists in the `extracted_entities`. If it does, update the corresponding field in the patient record to refresh basic patient information.
3. For list fields (`medications`, `allergies`, `symptoms`), iterate through each field. Check if the field exists in the `extracted_entities`. For each new item in the extracted list, add it to the patient record only if it's not already present to prevent duplicates.
4. For `contact_info`, check if it exists in the `extracted_entities`. If so, then merge it into the patient record's contact info to maintain updated contact details.

3. Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Implementing the Chat Function With Entity Memory

Extract patient information, update the entity store, and generate responses using both entity memory and conversation history.

A function named `chat()` is defined with parameters for `user_input` and `session_id` (defaulting to "patient_1") to handle patient conversations with entity tracking.

Inside the function:

1. Call `extract_patient_entities()` on the user input and store the result in a variable named `extracted` to identify any new patient information.
2. If entities were found (non-empty), call `update_entity_store()` with the `session_id` and `extracted` entities to merge the new information into the patient record.
3. Retrieve the current patient information by calling `get_entity_store()` with the `session_id` and store it in a variable named `patient_info` to pass the entity context to the `conversation_chain`.
4. Create a configuration dictionary with "configurable" as a key containing "session_id" mapped to the session ID value, and store this in a variable named `config` to enable session-specific memory retrieval.
5. Invoke the `conversation_chain` with a dictionary containing keys "input" (mapped to user input) and "patient_info" (mapped to patient information), along with the session configuration, to generate the assistant's reply. Store the result in a variable named `response`.

Hybrid Memory Systems

In longer conversations, a single memory strategy is often not enough. Some information, such as names, goals, or deadlines, needs to be remembered across many turns in the conversation. Other information, such as recent updates or follow-up questions, is relevant only to the current part of the conversation. To retain long-term details while also using recent context, an application uses a hybrid memory system.

A **hybrid memory system** combines multiple memory strategies. Each strategy is responsible for managing a different portion of the conversation history. This allows the model to use recent messages when responding, while still having access to important information mentioned earlier in the conversation.

Hybrid Memory Combinations

Different hybrid combinations are possible depending on the application and the type of information that needs to be retained. For example:

- **Buffer memory + entity memory** — useful when conversations are short, but specific facts such as user details or preferences must be retained reliably
- **Window memory + entity memory** — useful when conversational flow is important, but certain facts must persist regardless of how the conversation evolves

These are just a few common patterns. Hybrid memory systems can be extended or adapted to combine additional memory strategies as application requirements grow or change.

In this exercise, we focus on one specific hybrid configuration: window memory combined with summary memory. This combination is well-suited for long, structured conversations where both recent detail and long-term context matter.

Window memory keeps the most recent conversation exchanges exactly as they occurred. This allows the model to handle recent updates and follow-up questions where exact wording and immediate context matter.

Summary memory stores a compressed version of earlier conversation history. It replaces older messages with a short summary, without keeping the exact wording.

Together, window memory and summary memory allow conversations to continue without exceeding the model's token limit, while still preserving the context needed to generate useful responses.

How Hybrid Memory Works

In a hybrid setup, window memory and summary memory are maintained separately. They do not replace each other. Instead, both are used together whenever the model generates a response. Each user message follows the same architectural workflow.

1. New User Message Arrives

↓

2. Retrieve Existing Memory

└─ Load the summary memory (long-term context).

└─ Load the window memory (recent exchanges).

↓

3. Build Model Input

└─ The current user message

└─ Recent exchanges from the window memory

└─ The summarized history from the summary memory

↓

4. Send Context to Model

↓

5. Model Generates Response

↓

6. Store New Exchange in the Window Memory

└─ Add the user message.

└─ Add the model response.

↓

7. Enforce Context Window Size

└─ If the context window size in the window memory is within the limit, do nothing.

└─ If the context window size in the window memory exceeds the limit, proceed to summarization.

↓

8. Update Summary Memory (only if the window exceeds the limit)

└─ Summarize the oldest messages from the window memory.

└─ Append the summary to the summary memory.

└─ Remove the summarized messages from the window memory.

This workflow runs once for every user message.

Hybrid memory systems are commonly used in real-world assistants where conversations are long or ongoing, and important context must be preserved without exceeding memory limits.

To understand this approach in practice, we will build a project management assistant that uses hybrid memory. The assistant tracks a software project across 10 conversation turns. Early turns establish project setup, such as the project name, team members, and deadline. Later turns focus on sprint activities, including tasks, blockers, and bugs.

The system maintains:

- Window size: three exchanges
- Summary trigger: every six messages

We will test whether the assistant can recall both early project details (from the summary) and recent updates (from the window).

Instructions

1. Checkpoint 1 Enabled

1.

Implementing Summary Retrieval

Retrieve the saved summary for a session, or display a default message if no summary exists.

A function named `get_summary()` is already defined with a `session_id` parameter to identify which session's summary to retrieve. Inside the function:

1. Retrieve the summary for the given session from the `summary_store` dictionary and return it to access the condensed conversation history.
 2. If the session doesn't exist in the store, return the default string "No project history yet."
 - Use the `.get()` method on the `summary_store` dictionary to retrieve the summary for the given session.
 - Make sure to pass the `session_id` variable as the first argument to the `.get()` method to specify which session to retrieve.
 - Make sure to provide a default string value if the session doesn't exist.
2. Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Implementing Summary Creation

Generate a focused project summary by extracting key information from the conversation.

A function named `create_summary()` is defined with a `messages` parameter to handle the conversation history that needs to be condensed into a project summary. Inside the function:

1. Join all messages into a single string where each message includes its type and content on separate lines, storing this in a variable named `conversation_text` to create a readable format for the LLM to process.

2. A summary prompt is defined to instruct the model to create a concise project summary focusing on project goals, team members, decisions, tasks, blockers, deadlines, and action items, along with the conversation text included in the prompt.
 3. Use the summarizer model to generate a summary from the summary_prompt, store it in summary, and return only the final summary text to get the condensed project overview.
3. Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Implementing Hybrid Chat Function

Combine window memory for recent context and summary memory for older conversations, updating both as required.

A function named hybrid_chat() is defined with parameters for user_input and session_id to handle conversations using the hybrid memory approach. Inside the function:

1. Retrieve the current session data by calling get_session_history() and get_summary() with the session_id, storing the results in variables named history and current_summary respectively, to access both recent messages and condensed history.
2. Create a session configuration dictionary with the session_id and store it in a variable named session_config to identify this conversation session.
3. Invoke the conversation_chain with the user_input, current_summary, and session_config, storing the result in a variable named response to generate the AI's response using both memory types.
4. After getting the response, trim the conversation window to maintain only recent exchanges within the defined limit.

The function includes code to check if a summary update is needed (every SUMMARY_TRIGGER message) and updates the summary store accordingly to keep the condensed history current.

4. Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Testing Long-Term Memory Recall

Now let's test whether the hybrid system can recall information from early in the conversation. The team members and deadline were mentioned in Turns 2 and 3, which are now outside our 3-exchange window.

Invoke the `hybrid_chat()` function with the question and the `session_id`, storing the result in a variable named `long_term_response` to get the AI's answer based on both the summary and the window memory.

The `print()` statements are provided to display the test results.

5. Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Testing Recent Memory Recall

Let's also test whether the system can recall recent information from within the window. The QA bug was mentioned in Turn 10, which should still be in our three-exchange window.

Invoke the `hybrid_chat()` function with the question and the `session_id`, storing the result in a variable named `recent_response` to get the AI's answer based on the recent message history.

The `print()` statements are provided to display the test results.

6. Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Testing Hybrid Memory Integration

Finally, let's test whether the system can use both memory types together. This question requires both long-term context (from the summary) and recent details (from the window).

Invoke the `hybrid_chat()` function with the question and the `session_id`, storing the result in a variable named `hybrid_response` to get the AI's answer using both memory sources.

The `print()` statements are provided to display the test results.